Dependency Coverage Fuzzing: Black-Box Exploration by Controlled Input Interactions

Facundo Gomez Prates

October 1, 2026
Preprint — proof-of-concept evaluation

Abstract

We introduce *Dependency Coverage Fuzzing* (DepCover), a black-box input-selection criterion based on observed conditional interactions between input components. Its prototype constructs four-input contrasts that hold all other input bits fixed, records witnesses where toggling one bit changes the output only under a second bit's enabling value, and retains inputs associated with newly observed interactions. We evaluate an eight-bit synthetic target with three known interactions over 1,000 pseudorandom trials at each execution budget. At 64 distinct target executions, DepCover finds a mean of 2.960 of three interactions, compared with 2.007 for a feedback-free mutational black-box fuzzer and 0.655 for uniform sampling. Exhaustive enumeration verifies that the detector reports exactly the three expected interactions and none on an independent-output control. The evidence supports improved *interaction discovery per execution on this target*; it does not establish faster wall-clock execution, improved bug discovery, or superiority over instrumented fuzzers.

Keywords: black-box fuzzing; input interactions; behavioral coverage; controlled contrasts; empirical software testing.

1 Introduction

A black-box fuzzer observes a target through its inputs and outputs, without access to execution traces. A randomly selected input may reveal a new output, but it does not necessarily reveal *why* a mutation mattered: the effect of input component B may depend on the value of component A. Establishing this relationship requires matched observations in different contexts. This paper asks whether reserving executions for such matched observations improves the number of conditional interactions found under a fixed budget.

We call the resulting behavioral criterion *dependency coverage*. The metric counts conditional relationships supported by observable input-output contrasts. It is distinct from line, branch, or execution-path coverage. The method is black-box because the detector reads only input-output pairs; the experiment uses a synthetic target whose internals are known solely to establish ground truth. The directional notation $A \to B$ denotes an observed conditioning effect, not an internal data-flow edge or a causal direction in the program.

The contributions of this proof of concept are: (i) an operational witness rule for conditional input dependence; (ii) an implementation that schedules paired probes, caches observations, and maintains a small corpus; and (iii) a reproducible comparison at equal numbers of distinct target executions against uniform sampling and a mutational black-box baseline. The empirical claim is deliberately limited to the tested eight-bit target.

2 Related Work

AFL combines deterministic mutations, stochastic mutations, and feedback from instrumented execution; its technical documentation also describes corpus selection and input-region effect maps [1]. Our uniform and mutational baselines have no instrumentation and must not be interpreted as AFL comparisons. AFLFast studies seed scheduling under coverage feedback and demonstrates that the choice of which seeds to exercise materially affects exploration [2]. Delta debugging systematically isolates failure-inducing input differences [3]. DepCover similarly uses controlled differences, but its recorded object is an input interaction witnessed through ordinary program behavior rather than a minimized failing input. These works motivate more rigorous future comparisons; the present results do not establish novelty relative to every prior interaction-testing technique.

3 Model and Coverage Criterion

3.1 Observable target

Let $f : \{0,1\}^n \to \mathcal{O}$ be a deterministic target with an equality-testable outcome space $\mathcal{O}$. An outcome could include standard output, standard error, exit status, or a crash classification. The implementation in this study returns a three-bit tuple. For distinct positions A and B, let z fix all $n-2$ remaining positions. Write $x_{ab}(z)$ for the input with $A = a$, $B = b$, and all other positions equal to z.

Define the observed effect of toggling B under $A = a$ as

$$\Delta_B(a, z) = \mathbf{1}[f(x_{a0}(z)) \neq f(x_{a1}(z))]\,. \tag{1}$$

All four outcomes must refer to the same z. The prototype records a witness for $A \to B$ precisely when

$$\Delta_B(0, z) = 0 \quad \text{and} \quad \Delta_B(1, z) = 1. \tag{2}$$

Thus B has a visible effect only when $A = 1$ in this local context. This is a *witness criterion*, not an estimate

of a universal causal law. Output equality is exact in the present prototype; noisy outcomes would require repeated measurements and a statistical decision rule.

3.2 Coverage and orientation

Given an executed input set S, let $E(S)$ contain pairs for which all four members of at least one witness quartet occur in S. Dependency coverage is

$$\mathrm{DC}(S) = |E(S)|. \tag{3}$$

Only candidate pairs $A < B$ are examined. Orienting each edge from the smaller to the larger index yields an acyclic graph by construction. It is an indexing convention, not proof that A physically precedes B; an AND gate is symmetric, and reversing A and B also yields a conditional witness. In a real target the total number of discoverable edges is unknown. Ground truth E_{ref} is available here only because the benchmark is synthetic.

4 Algorithm

DepCover stores each distinct input and its outcome. On insertion, it checks whether the new input completes a quartet for any ordered candidate pair and records newly witnessed edges. A seed that produces a new edge is retained in a small *interesting-input* corpus. The current selection policy shuffles all $\binom{n}{2}$ candidate pairs, chooses a parent from that corpus, perturbs each other bit with probability 0.15, and requests all four inputs for the selected pair in shuffled order. Already executed inputs reuse cached outcomes. Any remaining budget after all pairs have been probed is spent on unseen inputs in random order.

Algorithm 1. Pair-guided black-box probing

1. Set $S \leftarrow \varnothing$, $E \leftarrow \varnothing$, $C \leftarrow \{0^n\}$.
2. Shuffle the ordered candidate pairs (A, B) with $A < B$.
3. For each pair, select $s \in C$, perturb positions other than A, B, and form $x_{00}, x_{01}, x_{10}, x_{11}$.
4. Query unseen quartet members until the budget is exhausted; cache outcomes in S.
5. On each new observation, inspect completed quartets and add newly witnessed edges to E; retain their discoverer in C.
6. If pairs are exhausted, query unseen inputs until exactly B distinct inputs have been executed.

The graph is a record of inferred relationships; it is *not* a recovered control-flow graph. At this stage the next pair is not prioritized by graph statistics. BFL, NDT, learned scoring, byte-level inputs, and crash-oriented scheduling are outside the evaluated implementation. With direct scans of all pairs after each distinct query, the prototype performs $O(Bn^2)$ pair checks and stores $O(B)$ observed inputs, excluding the cost of evaluating f. Its exhaustive pair enumeration is unlikely to scale to long files without pair selection and input-field abstractions.

5 Experimental Method

5.1 Target and controls

The positive target has eight input bits and three output bits:

$$f(x) = (x_0 \wedge x_1,\ x_2 \wedge x_3,\ x_4 \wedge x_5). \tag{4}$$

Bits x_6 and x_7 are distractors. Under the $A < B$ convention, the expected set is $E_{\mathrm{ref}} = \{(0,1),(2,3),(4,5)\}$. The negative control returns $f_0(x) = (x_1, x_3, x_5)$, in which the effect of an output bit does not depend on a second input bit under Eq. (2). Expected edges are used *only* by assertions and reporting, not by either input-selection policy.

5.2 Baselines and protocol

The *uniform* baseline samples distinct inputs by taking a pseudorandom permutation of all 256 possible inputs. The *mutation* baseline starts from zero, repeatedly chooses a uniformly random previously observed input, flips one, two, or three distinct bits with weights 3:2:1, and retains every distinct result. If duplicate proposals persist near saturation, it selects a random unseen input. It neither observes dependency edges for scheduling nor uses internal target coverage. Both baselines run the *same detector* as DepCover; only input selection changes.

For each budget $B \in \{32, 64, 96, 128, 192\}$, all methods execute exactly B distinct target inputs. Each budget uses 1,000 deterministic trials indexed by $t = 0, \ldots, 999$, with PRNG seeds $12345+1009t+k$ for DepCover ($k = 0$), mutation ($k = 1$), and uniform ($k = 2$). The primary measurement is the number of the three true edges discovered; we also count trials finding all three. Duplicate mutation proposals consume host computation but not target executions. Therefore the budget standardizes *target queries*, not wall-clock time or total candidate generation.

6 Results

Exhaustive execution of all 256 inputs yields exactly the three expected edges on the positive target and no edges on the independent-output control. Table 1 reports the budgeted runs.

At $B = 64$, DepCover exceeds the mutation baseline by 0.953 discovered edges on average and uniform sampling by 2.305. Approximate 95% normal intervals for differences of sample means across the independent PRNG streams are $[0.902, 1.004]$ and $[2.258, 2.352]$, respectively. These intervals characterize variation across randomized *selection orders on this fixed target*; they do not quantify uncertainty across programs. At $B = 192$, all methods find every edge in every trial, so the advantage disappears as the small input domain saturates.

The results support improved conditional-interaction discovery per distinct target execution on the tested program. No timing results or unique-bug counts were collected. The mutational baseline's repeated candidate generation is not included in the reported budget, and the guided method itself incurs bookkeeping overhead. No wall-clock efficiency claim follows from Table 1.

Table 1: Discovered true interactions across 1,000 trials per budget. Each cell gives mean edges out of three and the fraction of trials discovering all three. All methods share the same quartet detector and execute the stated number of distinct inputs.

	DepCover		Mutation baseline		Uniform baseline	
Budget	Mean / 3	Full / 1000	Mean / 3	Full / 1000	Mean / 3	Full / 1000
32	1.732	149	0.597	5	0.050	0
64	2.960	960	2.007	292	0.655	10
96	3.000	1000	2.889	894	2.188	386
128	3.000	1000	2.998	998	2.973	973
192	3.000	1000	3.000	1000	3.000	1000

7 Threats to Validity and Discussion

Target simplicity. Three independent AND gates create short, clean interactions. A bitwise XOR causes toggling B to alter the output under both values of A and is missed by Eq. (2), despite an algebraic interaction. Effects enabled when $A = 0$, higher-order gates, overlapping output dependencies, and unstable outcomes need more general criteria and repeated measurements.

Baseline scope. Uniform sampling is a deliberately simple reference. The mutational baseline is stronger but keeps every unique input and has no feedback-driven seed schedule. Neither is AFL, AFLFast, a grammar-aware black-box fuzzer, or a production parser fuzzer. The experiment cannot establish superiority over such systems. A stronger follow-up must measure several targets, varied seeds and formats, crash uniqueness, wall-clock cost, and equivalent test budgets.

Representation and scaling. The experiment assumes fixed-length binary inputs with exact outcome equality. Byte streams, protocols, and services require valid field mutations, meaningful outcome normalization, and timeout handling. The graph orientation is imposed by index, so it should not be interpreted as recovered internal causality. Full pair enumeration scales quadratically in the number of candidate input positions, while matching quartets becomes sparse in a large input space.

8 Conclusion

DepCover is a concrete black-box fuzzing criterion that treats locally witnessed conditional input effects as coverage. In a reproducible eight-bit proof of concept, paired probing discovers known dependencies in fewer target executions than a feedback-free mutational fuzzer and uniform sampling. This evidence concerns *observable interaction discovery*; tests of real programs, timing, and vulnerability detection remain necessary before stronger claims can be made.

Reproducibility. The accompanying `depcover_demo.py` uses only the Python standard library. For example, run `python3 depcover_demo.py -budget 64 -trials 1000`; repeat for all budgets in Table 1. The script performs an exhaustive positive and negative validation before reporting budgeted results.

References

[1] M. Zalewski, "Technical whitepaper for afl-fuzz," AFL documentation. https://android.googlesource.com/platform/external/AFLplusplus/+/b5d1a021/docs/technical_details.md.

[2] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based greybox fuzzing as Markov chain," in *Proc. ACM CCS*, 2016, pp. 1032–1043. doi: 10.1145/2976749.2978428.

[3] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Trans. Softw. Eng.*, vol. 28, no. 2, pp. 183–200, 2002. doi: 10.1109/32.988498.